

Лабораторная работа № 1 по курсу дискретного анализа: сортировка за линейное время

Выполнил студент группы М8О-207БВ-24 МАИ *Устинов Александр*.

Условие

1. Общая постановка задачи. Требуется разработать программу, осуществляющую ввод пар «ключ-значение», их упорядочивание по возрастанию ключа алгоритмом "Сортировка подсчетом" за линейное время и вывод отсортированной последовательности.
2. Вариант задания. Тип ключа: почтовые индексы. Тип значения: строки переменной длины (до 2048 символов).

Метод решения

Для решения задачи был использован алгоритм "Сортировка подсчетом" (Counting Sort), который позволяет сортировать целочисленные ключи за линейное время при условии, что диапазон ключей не слишком велик. Поскольку почтовые индексы представляют собой 6-значные числа, алгоритм эффективен для данной задачи. Для решения задачи была реализована своя структура данных

1. Создается массив счетчиков размером значения максимального ключа + 1.
2. Проход по входным данным для подсчета количества вхождений каждого ключа.
3. Вычисление префиксных сумм для определения позиций ключей в отсортированном массиве.
4. Создание выходного массива и заполнение его на основе подсчитанных позиций.

Архитектура программы:

`main.cpp`

- └ **Ввод данных** — чтение из `stdin` через `fgets()` <ключ>\t<значение>
- └ **Структура Entry** — ключ, ключ-строка, значение, длины строк
- └ **Класс EntryArray** — динамический массив записей с методами управления памятью
- └ **Функция counting_sort** — сортировка за $O(n + k)$
- └ **Вывод** — запись в `stdout` через `fwrite()` в исходном формате

Использованные источники:

- Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. "Алгоритмы: построение и анализ".
- https://en.wikipedia.org/wiki/Counting_sort

Описание программы

Программа реализована в 1 файле `main.cpp`

Описание основных типов данных:

1. `struct Entry` — структура для хранения пары ключ-значение, где ключ представлен как целое число и строка (для сохранения ведущих нулей), а значение — строка переменной длины.
2. `class EntryArray` — класс, инкапсулирующий динамический массив записей. Использует `malloc/realloc` для управления памятью, автоматически расширяется при необходимости.
3. `uint32_t*` — массив счётчиков для алгоритма сортировки подсчетом, где индекс соответствует ключу, а значение — количеству его вхождений.

Описание основных функций и методов:

1. `EntryArray::EntryArray` — конструктор, инициализирующий массив (выделение начальной памяти).
2. `EntryArray::~EntryArray` — деструктор, освобождающий память массива и всех строк в записях.
3. `EntryArray::push` — добавление записи в конец массива с проверкой и увеличением ёмкости при необходимости.
4. `EntryArray::reserve` — приватный метод увеличения ёмкости массива через `realloc()`.
5. `counting_sort` — функция, реализующая алгоритм сортировки подсчетом для массива записей. Принимает указатель на `EntryArray` и сортирует записи за $O(n+k)$.
6. `main` — основная функция, которая читает входные данные через `fgets()`, вызывает функцию сортировки и выводит результат через `fwrite()`.

Дневник отладки

1. Ключ `000000` выводился как `0`, так как хранился только как число.
Решение: добавлено поле `key_str` для сохранения исходного представления ключа.
2. Ввод завершался при первой пустой строке.
Решение: пустые строки теперь пропускаются через `continue`, а не завершают ввод.
3. Избыточное потребление памяти из-за оверхеда `std::string` и множественных выделений.
Решение: переход на C-стиль с `new/delete` и компактное хранение строк.

4. Вложенный цикл для сопоставления отсортированных ключей с записями.
Решение: Сортировка записей напрямую в функции `counting_sort` за $O(n + k)$.
5. Указатели на строки становились невалидными при расширении буфера ввода.
Решение: Выделение отдельной памяти для каждой строки через `new` сразу при чтении.
6. Цикл с декрементом `size_type` приводил к бесконечному циклу ($0 - 1 = \text{MAX_VALUE}$).
Решение: использование `signed`-типа `ssize_t` для обратного прохода в сортировке.
7. Массив счётчиков не обнулялся, содержал мусор.
Решение: инициализация нулём при выделении: `new uint32_t[max_val + 1]()`.
8. Медленный ввод/вывод при больших объёмах данных.
Решение: отключение синхронизации `ios::sync_with_stdio(false)` и `cin.tie(nullptr)`.

Тест производительности

Для проверки было проведено 4 теста: входные данные содержали 1000, 10000, 100000 и 1000000 записей. Время выполнения было измерено с помощью функции `std::chrono`. Результаты показали, что время выполнения растёт примерно линейно с увеличением количества записей, что соответствует теоретической сложностной оценке алгоритма $O(n + k)$.

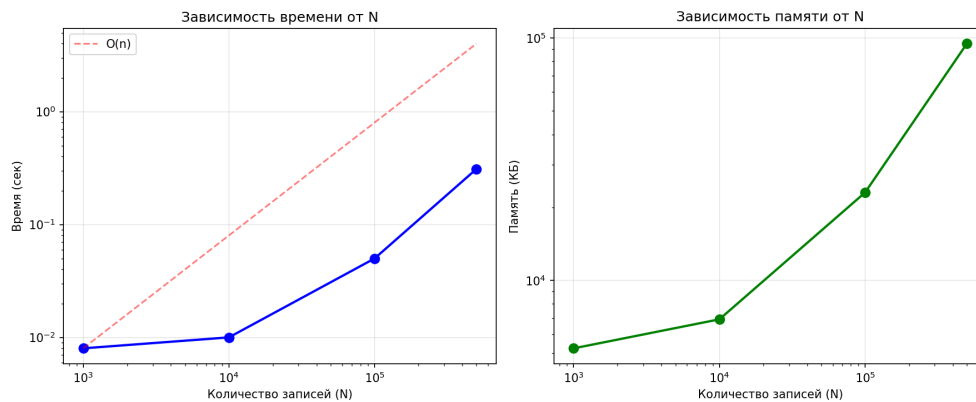


Рис. 1: Результаты тестирования производительности

График тестов сохранен в файле `benchmark.png`. Код для построения графика написан в `benchmark.py`

Недочёты

1. Массив счётчиков требует $O(k)$ памяти, где k — максимальный ключ. Для почтовых индексов (до 999999) это примерно 4 МБ.

Почему не исправлено: это фундаментальное ограничение алгоритма

2. При вводе нечисловых символов в ключе функция парсинга некорректно обрабатывает ввод.

Почему не исправлено: по ТЗ предполагается, что входные данные корректны.

Выводы

Сортировка подсчётом эффективна для сортировки целочисленных ключей в ограниченном диапазоне, когда требуется линейная сложность и стабильность. Типовые задачи: сортировка почтовых индексов, ID-номеров, символов, а также использование в качестве подпрограммы в поразрядной сортировке.

Сложность программирования — средняя. Алгоритм прост в реализации (около 30 строк кода), но требует внимательности при работе с граничными индексами и оптимизации памяти.

Возникшие проблемы: выход за границы массива в циклах, потеря ведущих нулей в ключах (решено хранением строкового представления), высокое потребление памяти (решено заменой `size_t` на `uint32_t`), квадратичная сложность при сопоставлении ключей (решено сортировкой записей напрямую). Все проблемы успешно устранены, программа соответствует требованиям ТЗ.